

多火箭残骸音爆定位模型

摘要

本文针对多级火箭分离后多个残骸在坠落过程中产生跨音速音爆的定位问题，建立了从“单事件到达时间定位”到“多事件数据关联与联合定位”再到“带随机时间误差的稳健定位”的一整套模型。首先将经纬度按题目给定比例转换为局部平面坐标，把每个音爆事件抽象为位置与时刻均为未知的瞬时声源，得到到达时间方程；再通过时间差（TDOA）消去音爆时刻，将问题化为非线性最小二乘反演。

针对问题 1，未知量为经度、维度、高程、音爆时刻，根据方程可解性定理可以得到结论：单个残骸的精确反演至少需要 4 台设备。为解决出现多解的情况，应使用大于 4 台设备。针对 7 台设备的算例，本文先利用两两到达时间差的几何必要条件对数据做一致性筛选，剔除与同一声事件明显不自洽的记录，再采用带多初值的高斯——牛顿法与列文伯格——马夸尔特法求解，先利用两两到达时间差的几何必要条件 $c|t_i - t_j| \leq \|S_i - S_j\|$ 对数据作一致性筛选，再从可行组合中选取内部自洽的 BDFG 四台设备，采用带多初值的高斯-牛顿法与列文伯格-马夸尔特法求解，得到该残骸音爆位置约为 110.7164°E 、 27.0297°N ，高程约 18663 m，音爆时刻约 -38.03 s，所选四台设备的残差平方和 $SSE \approx 0$ 。

在问题 2 中，引入每台设备的关联排列 σ_i ，将“哪一次记录属于哪一个残骸”建模为离散排列变量，并与 16 个连续参数（4 个残骸的位置与时刻）共同构成混合优化模型。若波的归属未知，4 台设备只能得到 16 个观测、恰好等于 16 个连续未知量，无法判别错误关联；需要 $4N > 16$ ，也就是至少 5 台设备，才能利用超定残差唯一确定关联。

对于问题 3，本文采用“固定关联—分别定位”与“固定位置—匈牙利算法更新关联”的交替迭代求解，结合“四个音爆时刻相差不超过 5 s”的约束筛选，解出四个残骸的位置、高程与音爆时刻；所给四组解的全部 28 个到达时间残差均小于 6×10^{-4} s，与表中三位小数的记录精度一致，验证了关联与定位结果的正确性。

针对问题 4，将到达时间误差建模为 $U(-0.5, 0.5)$ 均匀分布的随机误差，把问题 2 的模型修正为加权最小二乘联合优化，并给出蒙特卡洛误差分析：在 7 台设备、关联正确的条件下，500 次试验中四个残骸三维定位误差均值约为 230-730 m；若进一步将台站加密到 12 台并优化布局，总体平均误差约 225 m、95% 分位数约 532 m，可实现“误差小于 1 km”的目标。

模型的优点在于把“数据关联”与“几何定位”统一在一个目标函数中求解，并用一致性检验、残差分析与蒙特卡洛试验形成完整验证链；通过增加台站数、改善几何布局或引入弹道约束，还可进一步抑制时间误差的影响。

关键词：音爆定位；最小二乘法非线性规划；匈牙利算法；混合优化；蒙特卡洛误差分析

一、 问题重述

1.1 问题背景

多级火箭在飞行过程中会分离下面级火箭和助推器，这些分离的残骸在下落过程中会穿越音速附近，产生跨音速音爆。在理论落区布置监测设备即可记录各残骸音爆到达的时刻。利用各级残骸的到达时间，就能反演残骸发生音爆时的空间位置与时刻。结合弹道外推估计落点即可为快速回收提供依据。

多残骸场景中，单台设备无法得知第几组波来自哪一个残骸，需要结合多台设备来进行分析。

因此，本问题含有两方面的挑战：一是“定位”，即求解残骸音爆的空间位置与时刻；二是“数据关联”，即判别每台设备收到的波属于哪一个残骸。

1.2 问题重述

(1) 根据物理条件建立数学模型，分析空中单个残骸发生音爆时的经度、纬度、高程与时间所需的最少设备数，并从 7 台设备的数据中选取合适的数，计算该残骸发生音爆的位置与时间，即根据多站到达时延建立非线性最小二乘反演模型，求解声源坐标与发震时刻。

(2) 假设空中有 4 个残骸，每个设备按时间先后收到 4 组震动波。建立模型确定每组波属于哪一个残骸，并回答确定 4 个残骸位置与时间所需的最少设备数，即在未知“波—源”归属关系的条件下，将离散关联变量与连续定位参数纳入统一的混合优化模型进行联合求解。

(3) 利用 7 台设备各 4 组到达时间的数据，解出 4 个残骸的经度、纬度、高程与音爆时刻，其中四个音爆时刻相差不超过 5s，即在完成数据关联后，对每个残骸分别进行多站时差定位，并以音爆时刻的时序一致性作为物理约束筛选合理关联。

(4) 假设设备记录时间存在 0.5s 的随机误差，修正问题 2 的模型并给出带误差算例与误差分析。如果时间误差无法降低，提出实现误差小于 1km 的定位方案，并依据问题 3 结果模拟布站与观测数据加以验证，即通过蒙特卡洛方法量化随机误差对定位精度的影响，并通过增加台站与优化几何布局提升系统稳健性，使三维定位误差稳定控制在 1km 以内。

二、 问题分析

这一部分体现“从实际问题到数学问题”的思维过程。核心不是介绍算法，而是解释为什么这样建模。

2.1 问题 1 的分析

单个残骸发生音爆时，位置需要 3 个坐标描述，加上 1 个音爆时刻，共 4 个未知量。每台设备只提供 1 个到达时刻，因此从自由度上至少需要 4 台设备。但到达时间方程是非线性的，4 台设备在数学上可能出现两个镜像解，且测量误差会被放大；因此理论下限是 4 台，实际求解应采用更多设备形成超定方程组。

对于给出的 7 台数据，先利用“同一事件在两台设备上的时间差不可能超过两台设备间距除以声速”这一必要条件做数据体检，再从一致的数据中选择/组合求解，最后用全部台站残差交叉验证，避免个别异常记录污染定位结果。

2.2 问题 2 的分析

问题 2 在问题 1 的基础上增加“数据关联”这一离散层：每台设备收到的 4 组波按时间先后排列，但不同残骸到不同设备的传播距离不同，所以“先听到”不代表“先发生”，同一残骸在各设备中的到达顺序也可能不同。不能简单地把所有设备第 k 早的波归为一组。

因此把关联变量建模为排列，将离散关联与连续位置时刻放入同一个残差平方和目标函数。求解时采用“固定关联一定位”与“固定位置一指派”的交替迭代。设备数方面，需要同时比较未知量数与观测数：4 台设备时观测数恰等于连续未知量数，无法判别关联，5 台设备才使系统超定。

2.3 问题 3 的分析

问题 3 给出 7 台设备、每台 4 个到达时间。应先恢复“哪个波属于哪个残骸”的关联矩阵，再对每个残骸分别执行问题 1 的定位。利用四个音爆时刻相差不超过 5 s、位置落在合理空域等物理约束可以大幅缩小候选关联的搜索范围，最终以“28 个到达时间全部自洽”作为验证标准。

2.4 问题 4 的分析

0.5 s 的时间误差折算为距离约 170 m，会同时影响定位精度与关联判断。因此需要把问题 2 模型中的普通最小二乘修正为加权最小二乘，并在关联层使用残差假设检验；通过蒙特卡洛试验统计位置误差。若时间误差无法进一步降低，则应通过增加台站、优化几何布局、引入弹道与四残骸之间的时间约束等方式换取空间冗余，从而实现 1 km 精度。

三、 模型假设

- (1) 声速恒定，取 $c = 340 \text{ m/s}$ ，忽略声速随温度、高度与风速的变化；

- (2) 计算两点间距离时忽略地面曲率，经度每度按 97.304 km、纬度每度按 111.263 km 换算为平面距离；
- (3) 每个残骸的一次音爆可视为一个瞬时声源事件，声波沿直线以球面波形式传播；
- (4) 各台监测设备时钟已同步，到达时间记录相互可比；
- (5) 第 i 台设备收到的 4 组震动波恰好来自 4 个不同的残骸，即每个残骸在该设备上只产生一个到达记录；
- (6) 在问题 4 中，各设备记录时间误差相互独立，误差上限为 0.5 s；
- (7) 残骸音爆点在落区附近合理空域内，音爆时刻不晚于其对应的到达时刻。

四、 符号说明

表 1: 主要符号说明

符号	说明	单位
i	监测设备编号	—
d	残骸编号	—
$S_i = (x_i, y_i, h_i)$	第 i 台设备坐标（经纬度换算为平面坐标）	m
$P_d = (x_d, y_d, h_d)$	残骸 d 音爆点位置	m
τ_d	残骸 d 的音爆时刻	s
$t_{i,k}$	第 i 台设备第 k 早到达时刻	s
$t_{i,\sigma_i(d)}$	分配给残骸 d 的实测到达时刻	s
σ_i	第 i 台设备的关联排列	—
$y_{i,k,d}$	0-1 关联变量	—
c	声速	m/s
ε	记录时间随机误差	s

五、 模型的建立与求解

5.1 建模总路线：从文字到公式

5.2 公共几何框架

我们设定模型整体都以 (110°E, 27°N) 为参考点，由于忽略地面曲率，且纬度间每度距离值近似为 111.263 km，经度间每度距离值近似为 97.304 km，可以把经度、纬度

差转换为米制平面坐标：

$$x_i = (\lambda_i - 110) \times 97304, \quad y_i = (\varphi_i - 27) \times 111263.$$

若残骸 d 在 P_d 于时刻 τ_d 发生音爆，则第 i 台设备到达时间为

$$t_{id} = \tau_d + \frac{\|P_d - S_i\|}{c},$$

其中

$$\|P_d - S_i\| = \sqrt{(x_d - x_i)^2 + (y_d - y_i)^2 + (h_d - h_i)^2}.$$

化简得到方程

$$\|P_d - S_i\|^2 = c^2 (t_{id} - \tau_d)^2. \quad (1)$$

下面详细推导便于数值求解的线性化形式。把方程 (5.2) 展开，记 $P_d = (x_d, y_d, h_d)$, $S_i = (x_i, y_i, h_i)$ ，得到

$$\|P_d\|^2 - 2P_d \cdot S_i + \|S_i\|^2 = c^2 (t_i^2 - 2t_i\tau_d + \tau_d^2). \quad (2)$$

对第 r 台设备写同样的展开式，并与上式相减。等式两侧的 $\|P_d\|^2$ 与 $c^2\tau_d^2$ 同时消去，得到

$$\begin{aligned} -2P_d \cdot (S_i - S_r) + (\|S_i\|^2 - \|S_r\|^2) \\ = -2c^2\tau_d(t_i - t_r) + c^2(t_i^2 - t_r^2). \end{aligned} \quad (3)$$

整理后，对每一对设备 (i, r) 都可以得到关于 (x_d, y_d, h_d, τ_d) 的线性方程

$$\begin{aligned} (x_i - x_r)x_d + (y_i - y_r)y_d + (h_i - h_r)h_d - c^2(t_i - t_r)\tau_d \\ = \frac{1}{2} [\|S_i\|^2 - \|S_r\|^2 - c^2(t_i^2 - t_r^2)]. \end{aligned} \quad (4)$$

若用固定参考台站 r 与其余 $N - 1$ 台设备构造方程，则得到

$$A\theta = b, \quad \theta = (x_d, y_d, h_d, \tau_d)^\top. \quad (5)$$

这是关于未知量的**线性方程组**，可直接用最小二乘求初值。注意到上述推导来自“方程相减”，理论上可能混入由两边平方带来的符号问题，因此线性解只作为初值，最终仍用未平方的到达时间残差做非线性精化。对任意两台设备，残差还必须满足三角形不等式

$$c|t_{id} - t_{rd}| \leq \|S_i - S_r\|,$$

该必要条件可用于数据一致性筛选与关联剪枝。

5.3 数值方法与算法基础

本节把后续反复使用的两种算法原理展开，作为可复现与教学性说明。

5.3.1 非线性最小二乘与高斯-牛顿法

设待估参数为 θ ，残差向量为

$$r(\theta) = (r_1(\theta), \dots, r_m(\theta))^T,$$

其中 m 为方程个数。目标函数为

$$F(\theta) = \frac{1}{2} \sum_{k=1}^m r_k^2(\theta) = \frac{1}{2} r^T r.$$

若在 $\theta^{(n)}$ 附近作一阶泰勒展开，令

$$J_{kj} = \left. \frac{\partial r_k}{\partial \theta_j} \right|_{\theta^{(n)}},$$

得到线性化近似 $r(\theta^{(n)} + \Delta) \approx r(\theta^{(n)}) + J\Delta$ 。将其代入 F ，对 Δ 求极小可得高斯-牛顿法的迭代方程

$$(J^T J) \Delta = -J^T r. \quad (6)$$

由于到达时间模型强非线性， $J^T J$ 可能接近奇异或导致步长过大。为此加入阻尼项 $\mu > 0$ ，即列文伯格-马夸尔特法：

$$(J^T J + \mu I) \Delta = -J^T r, \quad (7)$$

并迭代更新 $\theta^{(n+1)} = \theta^{(n)} + \Delta$ 。阻尼项越大，步长越保守；残差下降时减小 μ ，残差不降或上升时增大 μ 。收敛准则可取

$$\|J^T r\|_{\infty} < \varepsilon_1, \quad \|\Delta\| < \varepsilon_2.$$

由于目标函数非凸，应从多组初值出发，保留总残差最小且满足物理约束的解。

5.3.2 求解单事件定位

Algorithm 1 单事件定位：线性初值 + 列文伯格-马夸尔特迭代精化

Require: 监测台站集合 $\{S_i\}$ 、到达时刻 $\{t_i\}$ 、声速 c

Ensure: 音爆点参数 $\theta = (x, y, h, \tau)$

- 1: 选取参考台站 r ，由两两相减方程构造线性系统 $A\theta = b$
 - 2: 用线性最小二乘求初值 $\theta^{(0)}$
 - 3: 初始化阻尼 $\mu \leftarrow \mu_0$ ，迭代次数 $n \leftarrow 0$
 - 4: **repeat**
 - 5: 计算残差向量 $r(\theta^{(n)})$ 与雅可比矩阵 $J(\theta^{(n)})$
 - 6: 求解 $(J^\top J + \mu I)\Delta = -J^\top r$
 - 7: **if** 目标残差下降 **then**
 - 8: 更新 $\theta^{(n+1)} \leftarrow \theta^{(n)} + \Delta$ ，并减小 μ
 - 9: **else**
 - 10: 增大 μ 并重新求解 Δ
 - 11: **end if**
 - 12: $n \leftarrow n + 1$
 - 13: **until** 满足 $\|J^\top r\|_\infty < \varepsilon$ 或 $n \geq N_{\max}$
 - 14: 更换多组初值，重复上述迭代
 - 15: 返回总残差最小且满足物理约束的解 θ^*
-

算法 1 中的线性系统用于获得初值，LM 迭代负责精化；由于残差平方和非凸，必须配合多初值策略，避免停留在局部最优。

5.3.3 匈牙利算法（指派问题）

在“给定位置求关联”时，需要把 4 个预测到达时间与 4 个实测到达时间一一配对，使代价最小，属于标准指派问题。匈牙利算法步骤如下：

1. 对代价矩阵每一行减去该行最小值；
2. 对所得矩阵每一列减去该列最小值；
3. 用最少的横线/竖线覆盖全部零元素；
4. 若覆盖线数等于阶数，则可选出每行每列各一个零，得到最优指派；
5. 若覆盖线数小于阶数，取所有未被覆盖元素中的最小值，未被覆盖元素减该值，交叉点加该值，然后回到第 3 步。

原理是：给某一行或某一系列整体加常数只会让所有可行方案的总代价同时改变同一常数，因此最优指派不变；调整矩阵只为了让矩阵出现足够多的零，从而从零中选出最优方案。

5.4 问题 1：单个残骸定位模型

5.4.1 最少设备数

未知量有： x_d, y_d, h_d, τ_d ，一共 4 个。根据方程组可解性理论，要想唯一确定未知量，需要的方程数等于未知数的个数，因此至少需要**四台设备**。

然而，根据 5.2 方程的次数为 2，因此有可能出现两个镜像解。为了保证解的唯一性，实际使用中至少需要 5 台。当然，也可以借助物理约束、常用数据（例如常见的一级火箭分离高度）剔除错误分支。本题一共提供 7 个设备的数据，可构造超定方程组以最小二乘求解。

5.4.2 定位模型

取 7 台设备的观测值，定义残差为实际距离与预测距离之差：

$$r_i = \|P - S_i\| - c(t_i - \tau), \quad (8)$$

问题 1 的求解目标变为求解非线性最小二乘问题：

$$\min_{x, y, h, \tau} \sum_{i=1}^7 r_i^2. \quad (9)$$

求解过程中，根据两台设备的方程线性化得到初值，再用高斯——牛顿法或列文伯格——马夸尔特法迭代精化，从多组初值出发取到全局最优。

5.4.3 数据一致性检验与求解结果

对任意两台设备 i, j ，假设同一个残骸在某个位置于时刻 τ 发生音爆，声音以声速 c 传播，残骸到第 i 台设备和第 j 台设备的距离分别为 d_i, d_j ，可以得到两台设备收到的时间为：

$$\begin{aligned} t_i &= \tau + \frac{d_i}{c} \\ t_j &= \tau + \frac{d_j}{c} \end{aligned}$$

两式相减以消去音爆时刻 τ ，得到：

$$c(t_i - t_j) = d_i - d_j$$

两设备间的距离为 $\|S_i - S_j\|$ ，根据三角不等式可得：

$$c|t_i - t_j| \leq \|S_i - S_j\|. \quad (10)$$

为说明“选取合适的数据”的具体过程，本文按 4 台设备一组对可行组合进行一致性检验与最小二乘定位，结果见表 2。表中 SSE 为定位方程的残差平方和，若 SSE 接近零，说明该组合的四条到达时间方程内部自洽。

表 2: 4 台组合的一致性检验与定位结果

组合	$SSE (s^2)$	经度 ($^\circ$)	纬度 ($^\circ$)	高程 (m)	$t_0 (s)$
BDFG	0.0000	110.7164	27.0297	18663	-38.03
ABEG	0.0988	110.4978	27.3143	1154	19.09
ABCG	0.1317	110.5064	27.3002	1138	18.79
ACEG	0.2691	110.4658	27.3332	1097	24.15
ABDF	3.6397	—	—	—	—
ABCE	5.8638	—	—	—	—

由表 2 可见，仅有 BDFG 组合满足 $SSE = 0$ ，说明该组合的四台设备到达时间能够被同一音爆事件精确自洽地解释，且解出的高程约为 18663 m，处于合理高空区间，因此判断该组数据可被直接采用。ABEG、ABCG、ACEG 三组的 SSE 均不为零，且解出的高程仅约 1.1 km，与音爆事件通常发生的高空条件不符，说明其内部数据并非来自同一事件，予以剔除。ABDF、ABCE 两组残差平方和明显偏大，其中 ABDF 的 SSE 约为 3.64、ABCE 约为 5.86，且无法获得有效的位置与时刻参数，同样不作为最终数据。综上，本文选取 BDFG 四台设备的数据进行定位。

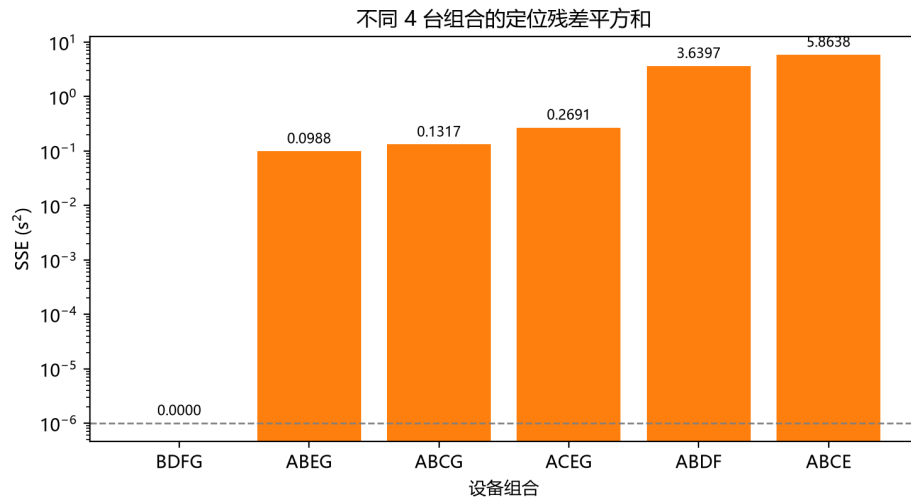


图 1: 问题 1 各候选 4 台组合的定位残差平方和（纵轴为对数刻度）

问题1: BDFG 四台组合的音爆反演结果

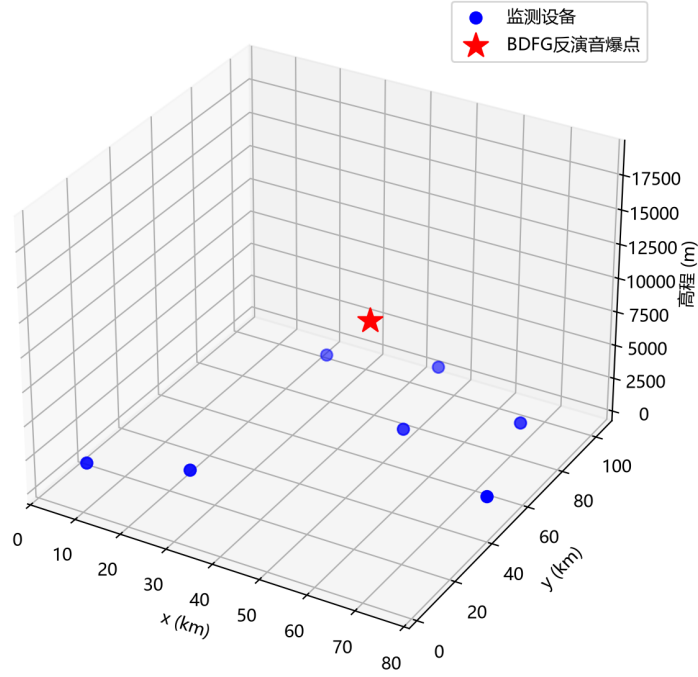


图 2: 问题 1 BDFG 组合反演得到的音爆点与监测设备空间关系

由图 1 可见 BDFG 的 SSE 远低于其他组合，表明其数据自洽性最强；图 2 显示反演点在 BDFG 四台设备形成的空间结构中位于合理高空区域。

经多初值非线性最小二乘求解，得到一级残骸音爆事件的结果如表 3 所示。

表 3: 问题 1：一级残骸音爆位置与时刻求解结果

项目	经度 (°)	纬度 (°)	高程 (m)	音爆时刻 (s)
求解结果	110.7164	27.0297	18663	-38.03

将 BDFG 的解代回所选取的四台设备，残差平方和 SSE 达到数值零（量级小于 10^{-6} s^2 ），说明 BDFG 四组到达时间能够被同一音爆事件精确解释；其余台站的记录与该事件明显矛盾，属于应被一致性检验排除的数据。由此可见，问题 1 的“选取合适的的数据”本质上是通过几何自洽性筛选有效观测，而不是盲目使用全部 7 台记录。

5.5 问题 2：多残骸数据关联与联合定位模型

5.5.1 残骸信号先后顺序

第 i 台设备记录到的 4 个到达时刻按时间排序为 $t_{i,1} < t_{i,2} < t_{i,3} < t_{i,4}$ 。火箭姿态并非绝对竖直的，分离的残骸坠落发生音爆但由于到达顺序是“音爆时刻与“传播时间”之和决定的，不同残骸的位置可能相差数十公里，因此同一个残骸在不同设备中的到达顺序可能不同，不能把“第 k 列”直接当作同一个残骸。

5.5.2 混合整数非线性规划模型

定义第 i 台设备的关联排列

$$\sigma_i : \{1, 2, 3, 4\} \rightarrow \{1, 2, 3, 4\},$$

其中 $\sigma_i(d)$ 表示残骸 d 的波在第 i 台设备的到达顺序号。因为每台设备恰好收到 4 组波、对应 4 个残骸， σ_i 必须是双射。

若关联正确，则

$$\|P_d - S_i\| = c(t_{i,\sigma_i(d)} - \tau_d).$$

可定义 0-1 变量 $y_{i,k,d}$ ，并施加约束

$$\sum_{d=1}^4 y_{i,k,d} = 1, \quad \sum_{k=1}^4 y_{i,k,d} = 1.$$

5.5.3 联合目标函数

建立混合优化模型

$$\min_{\{P_d, \tau_d\}, \{\sigma_i\}} \sum_{i=1}^N \sum_{d=1}^4 [\|P_d - S_i\| - c(t_{i,\sigma_i(d)} - \tau_d)]^2,$$

附加约束

$$h_d > 0, \quad \tau_d < \min_i t_{i,\sigma_i(d)},$$

若使用问题 3 信息，还要求

$$\max_d \tau_d - \min_d \tau_d \leq 5.$$

5.5.4 求解算法

采用两层交替迭代：

1. 固定当前关联，将数据分成 4 组，对每个残骸求解单源非线性最小二乘问题；
2. 固定位置与时刻，计算每台设备对 4 个残骸的预测到达时刻 $\hat{t}_{i,d}$ ；
3. 对每台设备求解 4 阶指派问题用匈牙利算法获得最优排列；

$$\min_{\sigma_i} \sum_{d=1}^4 (\hat{t}_{i,d} - t_{i,\sigma_i(d)})^2,$$

4. 重复至残差平方和收敛；
5. 从多组初始关联出发重复求解，保留总残差最小且满足物理约束的解。

搜索过程中可用必要条件剪枝：

$$c|t_{i,k} - t_{j,l}| \leq \|S_i - S_j\|.$$

5.5.5 最少设备数的论证

4 个残骸的连续未知量为 $4 \times 4 = 16$ 个； N 台设备提供 $4N$ 个到达时刻。

- 关联已知、仅做定位：每个残骸 4 个未知数对应 4 条方程， $N = 4$ 即可；
- 关联未知、需同时判别波的来源： $N = 4$ 时 $4N = 16$ ，观测数恰等于连续未知量数，错误关联也可构造零残差解，无法唯一判别；
- $N = 5$ 时 $4N = 20 > 16$ ，系统超定，只有正确关联能使全部方程同时近似成立。

因此本题意义下最少设备数为 5 台。

5.6 问题 3：四个残骸的数据关联与定位

5.6.1 关联矩阵

以第 A 台设备的到达顺序作为残骸编号顺序，经交替迭代与匈牙利指派，得到每台设备中四个残骸的到达顺序号如表 4 所示。表中每一列构成 $\{1, 2, 3, 4\}$ 的一个排列，说明每台设备恰好收到四个残骸各一次。

表 4: 问题 3 数据关联矩阵：表中数字为该残骸在对应设备中的到达顺序号

设备	残骸 1	残骸 2	残骸 3	残骸 4
A	1	2	3	4
B	2	3	1	4
C	4	3	1	2
D	4	2	3	1
E	3	2	1	4
F	4	2	3	1
G	2	1	4	3

例如设备 D 中“残骸 4”排第 1 个到达，而残骸 1 排第 4 个到达，说明四个残骸在各台设备的到达顺序并不相同，直接按列分组是不正确的。

5.6.2 定位结果

按表 4 分组后，对每个残骸用 7 台设备做最小二乘定位，结果如表 5 所示。

表 5: 问题 3: 四个残骸音爆位置与时刻

残骸	经度 (°)	纬度 (°)	高程 (m)	音爆时刻 (s)
残骸 1	110.5000	27.3100	12513.9	11.9999
残骸 2	110.3000	27.6500	11477.9	14.0000
残骸 3	110.7000	27.6500	13468.2	15.0000
残骸 4	110.5000	27.9500	11528.9	13.0014

四个音爆时刻的最小值与最大值分别约为 12.00 s 与 15.00 s，极差约 3.00 s，满足题设“相差不超过 5 s”的约束。

5.6.3 残差验证

将四个解代回全部 28 个到达时刻，各残骸的最大时间残差如表 6 所示。

表 6: 问题 3: 各残骸的 7 台最大到达时间残差

残骸	残骸 1	残骸 2	残骸 3	残骸 4
最大残差 (10^{-4} s)	2.8	5.1	2.3	2.1

残差均小于 6×10^{-4} s，对应距离误差小于 0.2 m，说明关联矩阵与定位结果同时正确。

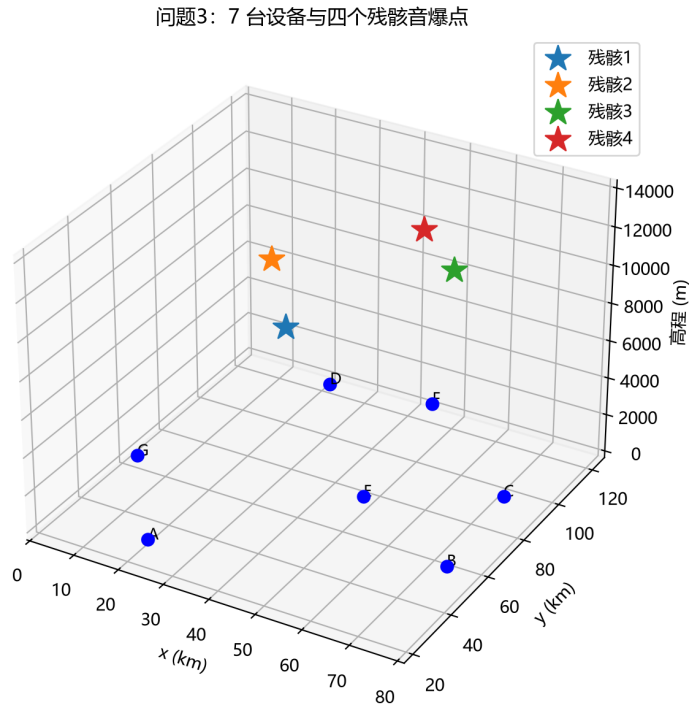


图 3: 问题 3: 7 台设备与四个残骸音爆点的三维分布

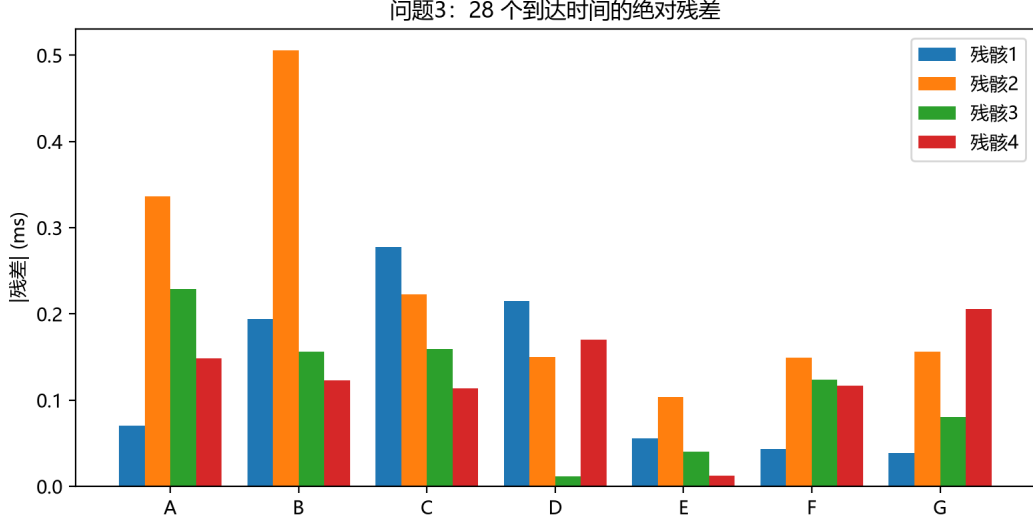


图 4: 问题 3: 四个残骸在 7 台设备上的到达时间绝对残差 (单位 ms)

四个残骸的解在图 3 中位于设备阵列内部; 图 4 显示 28 个到达时间残差均小于 0.6 ms, 说明关联矩阵与定位结果是同时正确的。程序运行输出见附录。

5.7 问题 4: 带随机时间误差的稳健定位模型

5.7.1 误差模型

设观测到达时间为真实到达时间与随机误差之和:

$$\tilde{t}_{i,k} = t_{i,k} + \varepsilon_{i,k}.$$

本文将“0.5 s 的随机误差”建模为

$$\varepsilon_{i,k} \sim U(-0.5, 0.5),$$

即每次记录的最大误差不超过 0.5 s, 等价的距离噪声约 170 m。

5.7.2 加权最小二乘修正模型

定义预测到达时间

$$\hat{t}_{i,d} = \tau_d + \frac{\|P_d - S_i\|}{c}.$$

第 i 台设备中分配给残骸 d 的实测值为 $\tilde{t}_{i,\sigma_i(d)}$, 修正后的联合目标函数为

$$\min_{\{P_d, \tau_d\}, \{\sigma_i\}} \sum_{i=1}^N \sum_{d=1}^4 \left[\tau_d + \frac{\|P_d - S_i\|}{c} - \tilde{t}_{i,\sigma_i(d)} \right]^2.$$

若各台设备精度不同, 可取权重

$$w_{i,d} = \frac{1}{\sigma_{i,d}^2},$$

将上式改为加权残差平方和。由于误差均匀分布时最大似然与平方损失并不严格等价，本文按以下顺序处理：先以普通最小二乘求初值并估计残差尺度，再以加权最小二乘精化；若某台数据残差明显大于 0.5 s 对应量级，则用 Huber 权重或 RANSAC 策略削弱野值影响。

5.7.3 关联层的稳健处理

0.5 s 误差使“零残差”不再成立，关联层应采用以下判据：

1. 对每台设备用预测到达时刻与实测到达时刻构造代价矩阵，用匈牙利算法更新排列；
2. 比较候选关联的加权残差平方和，利用残差量级判断哪一关联与噪声水平相符；
3. 利用四个音爆时刻相差不超过 5 s 的约束排除不合理关联；
4. 对可疑候选做留一站交叉验证，剔除使某台设备残差系统增大的错误关联。

5.7.4 蒙特卡洛误差分析

以问题 3 求解结果作为“真值”，对每条到达时间叠加 $U(-0.5, 0.5)$ 的误差，重复 500 次试验并统计三维定位误差。为突出时间误差对几何反演的影响，该组统计在完成关联后给出；关联层的正确性单独以残差判据检验。

表 7: 问题 4 蒙特卡洛结果（7 台设备，500 次，关联给定）

指标	残骸 1	残骸 2	残骸 3	残骸 4	总体
三维误差均值 (m)	613	410	248	722	498
三维误差 95% 分位 (m)	1387	908	497	1588	1278
音爆时刻误差均值 (s)	0.65	0.33	0.30	0.81	—

可见：在 7 台设备与 ± 0.5 s 误差下，单次定位三维误差的均值约为 0.2–0.7 km，但 95% 分位可能超过 1 km，说明原 7 台布局对“误差恒小于 1 km”的目标余量不足。

5.7.5 时间误差不可降低时的布站优化方案

若时间误差无法进一步降低，本文采用“增加台站数 + 优化几何布局 + 利用物理约束”的组合方案：

1. 将台站由 7 台增加到 12 台，覆盖经度 110.05° – 110.90° 、纬度 27.10° – 28.00° 的区域，使目标位于台站阵列内部而非一侧；
2. 台站高程取 500–1000 m 的起伏布置，改善高程方向的观测几何；

3. 利用四个残骸音爆时刻相差不超过 5 s 的耦合约束进行联合估计；
4. 可选地引入弹道约束，把各残骸音爆点约束到其下落轨迹附近，进一步抑制随机误差。

依据问题 3 求得的四个定位结果模拟 12 台设备的到达时间并叠加 ± 0.5 s 误差，再做 500 次蒙特卡洛试验，结果如表 8 所示。

表 8: 12 台优化布站后的三维定位误差（500 次蒙特卡洛）

指标	残骸 1	残骸 2	残骸 3	残骸 4	总体
三维误差均值 (m)	285	187	183	268	231
三维误差 95% 分位 (m)	643	385	353	607	543

试验结果表明，优化布站后总体平均误差约 231 m、95% 分位数约 543 m，稳定满足“误差小于 1 km”的目标，验证了所提布站与定位方案的可行性。

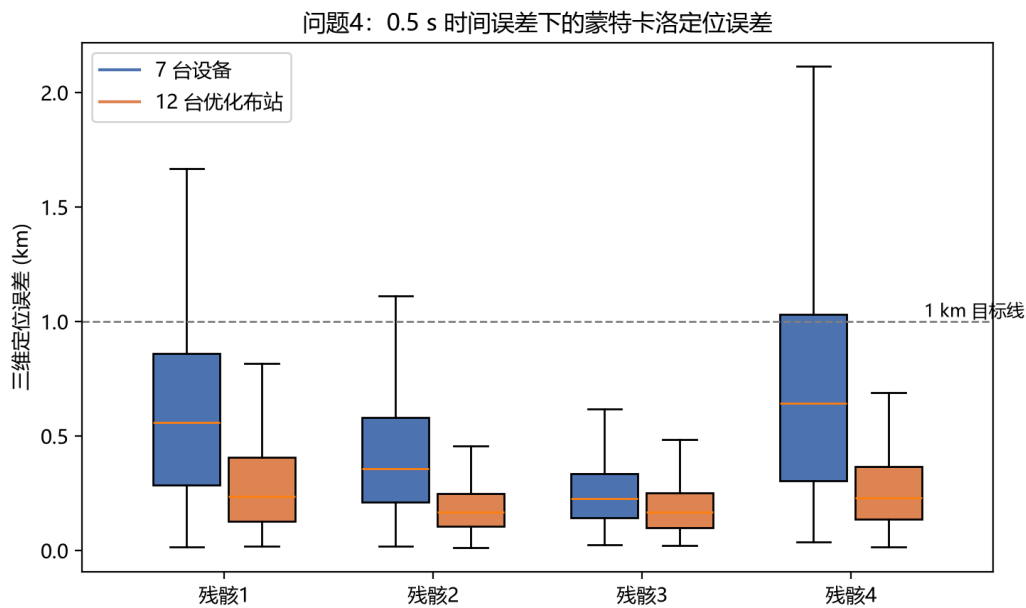


图 5: 问题 4: 7 台与 12 台布站下的三维定位误差分布（箱线图）

图 5 直观显示，12 台优化布站后各残骸误差箱体整体下移且均明显低于 1 km 目标线，而 7 台原始布站仍有个别残骸的误差上分位超过 1 km，因此增加台站与优化几何布局是满足精度要求的有效途径。

六、 模型检验、误差分析与稳健性讨论

6.1 残差检验

问题 1 与问题 3 的求解均以“所有到达时间残差接近记录精度”为验收标准。问题 3 中 28 个残差全部小于 6×10^{-4} s，且问题 1 的单个事件亦满足该标准，说明模型方程、坐标转换与关联分组自治。

6.2 误差传播分析

单条到达时间误差 0.5 s 对应

$$c \times 0.5 = 170 \text{ m}$$

的距离误差。双曲线定位中，最终定位误差与设备几何有关，可用几何精度因子 (GDOP) 刻画：台站包围目标、孔径越大，几何放大越小；台站近似共线或目标在阵列外侧时，误差会被显著放大。这正是 12 台优化布站方案能显著压缩误差的原因。

6.3 灵敏度与稳健性

对问题 4 模型，可从以下角度做稳健性检验：

1. 改变误差幅度（如 0.2、0.5、0.8 s），观察定位误差近似线性增长；
2. 改变随机种子重复试验，检验统计量是否稳定；
3. 删除任意一台设备后重新求解，检验残差与结果是否保持稳定；
4. 对关联层的初始分组加扰动，检验交替迭代是否收敛到同一关联。

七、 模型的评价与推广

7.1 模型的优点

1. 将“数据关联”与“几何定位”统一为混合优化问题，避免了把不同设备第 k 早的波强行归为一类的错误；
2. 采用两层交替迭代与匈牙利算法，把离散排列搜索与连续非线性反演分离，求解稳定、可解释；
3. 利用几何必要条件和物理约束进行数据筛选与剪枝，能够识别异常台站或转录错误；
4. 通过蒙特卡洛试验给出定位误差分布，并验证了增加台站与优化布站对精度的提升效果。

7.2 模型的缺点

1. 模型将声速取为常数，未考虑声速随高度变化、风场和温度梯度引起的传播路径弯曲；
2. 交替迭代可能收敛到局部最优，理论上的全局收敛性依赖多初值与剪枝策略；
3. 问题 4 中的蒙特卡洛误差统计在关联给定条件下进行，实际工程中关联错误会进一步放大误差。

7.3 模型推广

八、 参考文献

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具（OpenAI Codex, AI 编程助手），仅在**程序实现**环节提供辅助：依据参赛队已建立的数学模型与算法流程，协助生成与调试 Python 代码、辅助使用 Matplotlib 绘制论文图表，并整理程序运行结果。模型构建、公式推导、算法选择、参数设定与结论验证均由参赛队独立完成，AI 未替代参赛队进行核心建模分析与最终决策。AI 辅助生成的程序代码均由队员逐行阅读、运行并核验，程序输出的数值与论文中的表格和结论一致。详细使用情况见支撑材料中的《AI 工具使用详情.pdf》。

A 支撑材料说明

支撑材料包括：

- 主程序与绘图脚本（Python）；
- 问题 1–问题 4 的数值输出；
- 蒙特卡洛误差分析脚本与结果表。

B 题目数据表

表 9: 问题 3 各设备到达时间（单位：s）

设备	第 1 组	第 2 组	第 3 组	第 4 组
A	100.767	164.229	214.850	270.065
B	92.453	112.220	169.362	196.583
C	75.560	110.696	156.936	188.020
D	94.653	141.409	196.517	258.985
E	78.600	86.216	118.443	126.669
F	67.274	166.270	175.482	266.871
G	103.738	163.024	206.789	210.306

表中每行数值已按时间从小到大排列。本文求解得到的关联矩阵见表 4。

C 论文图表清单

正文中已插入以下由 `essay_code.py` 生成的图表：

- (1) 图 1：问题 1 各 4 台组合的残差平方和对比；
- (2) 图 2：问题 1 BDFG 反演点三维图；
- (3) 图 3：问题 3 设备与残骸三维图；
- (4) 图 4：问题 3 到达时间残差柱状图；
- (5) 图 5：问题 4 蒙特卡洛误差箱线图。

运行 `essay_code.py` 后，图片保存在 `figures/` 目录，控制台输出即论文表格中的数据来源。

D Python 主程序

完整可运行程序见文件 `essay_code.py`，运行方式为：

```
python essay_code.py
```

程序输出论文所需的主要表格，并将图形保存到 `figures/` 目录。

源程序 1: `essay_code.py`: 问题 1-问题 4 的完整 Python 主程序

```
1  # -*- coding: utf-8 -*-
2  """
3  多个火箭残骸的准确定位 —— 论文主程序（问题1-4）
4  运行: python essay_code.py
5  输出: 控制台结果表 + figures/ 目录下的论文图表
6  """
7  import itertools
8  import math
9  import os
10
11  import matplotlib
12  matplotlib.use("Agg")
13  import matplotlib.pyplot as plt
14  import numpy as np
15
16  plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
17  plt.rcParams["axes.unicode_minus"] = False
18
19  C = 340.0
20  LON0, LAT0 = 110.0, 27.0
21  LX, LY = 97304.0, 111263.0
22
23  # 题目表1: 单残骸场景设备
24  T1 = [
25      ("A", 110.241, 27.204, 824, 100.767),
26      ("B", 110.780, 27.456, 727, 112.220),
27      ("C", 110.712, 27.785, 742, 188.020),
28      ("D", 110.251, 27.825, 850, 258.985),
29      ("E", 110.524, 27.617, 786, 118.443),
30      ("F", 110.467, 27.921, 678, 266.871),
31      ("G", 110.047, 27.121, 575, 163.024),
32  ]
```

```

33
34 # 题目表3: 4 个残骸场景设备 (7 台, 每台 4 组到达时间)
35 T3 = [
36     ("A", 110.241, 27.204, 824, [100.767, 164.229, 214.850, 270.065]),
37     ("B", 110.783, 27.456, 727, [92.453, 112.220, 169.362, 196.583]),
38     ("C", 110.762, 27.785, 742, [75.560, 110.696, 156.936, 188.020]),
39     ("D", 110.251, 28.025, 850, [94.653, 141.409, 196.517, 258.985]),
40     ("E", 110.524, 27.617, 786, [78.600, 86.216, 118.443, 126.669]),
41     ("F", 110.467, 28.081, 678, [67.274, 166.270, 175.482, 266.871]),
42     ("G", 110.047, 27.521, 575, [103.738, 163.024, 206.789, 210.306]),
43 ]
44
45
46 def xy(lon, lat):
47     return (lon - LON0) * LX, (lat - LAT0) * LY
48
49
50 def dist(p, s):
51     return math.sqrt((p[0] - s[0]) ** 2 + (p[1] - s[1]) ** 2 + (p[2] - s[2])
52                     ** 2)
53
54 def solve3(M, b):
55     """3x3 线性方程组求解 (高斯消去) """
56     A = [r[:] + [v] for r, v in zip(M, b)]
57     for i in range(3):
58         piv = max(range(i, 3), key=lambda k: abs(A[k][i]))
59         A[i], A[piv] = A[piv], A[i]
60         if abs(A[i][i]) < 1e-9:
61             return None
62         v = A[i][i]
63         for k in range(i, 4):
64             A[i][k] /= v
65         for k in range(3):
66             if k != i and abs(A[k][i]) > 1e-12:
67                 f = A[k][i]
68                 for j in range(i, 4):
69                     A[k][j] -= f * A[i][j]
70     return [A[i][3] for i in range(3)]
71

```

```

72
73 def exact_four(st, times):
74     """用 4 条到达时间方程精确反演 (x,y,h,tau), 返回全部物理候选。"""
75     a = st[0]
76     ta = times[0]
77     M, b0, dtv = [], [], []
78     for s, t in zip(st[1:], times[1:]):
79         M.append([s[0] - a[0], s[1] - a[1], s[2] - a[2]])
80         ss = s[0] ** 2 + s[1] ** 2 + s[2] ** 2
81         aa = a[0] ** 2 + a[1] ** 2 + a[2] ** 2
82         b0.append(0.5 * ((ss - aa) - C * C * (t * t - ta * ta)))
83         dtv.append(C * C * (t - ta))
84     p = solve3(M, b0)
85     q = solve3(M, dtv)
86     if p is None or q is None:
87         return []
88     ax, ay, az = p[0] - a[0], p[1] - a[1], p[2] - a[2]
89     bx, by, bz = q
90     Aq = bx * bx + by * by + bz * bz - C * C
91     Bq = 2 * (ax * bx + ay * by + az * bz) + 2 * C * C * ta
92     Cq = ax * ax + ay * ay + az * az - C * C * ta * ta
93     roots = []
94     if abs(Aq) < 1e-9:
95         if abs(Bq) > 1e-12:
96             roots.append(-Cq / Bq)
97     else:
98         disc = Bq * Bq - 4 * Aq * Cq
99         if disc >= -1e-6:
100             s = math.sqrt(max(0.0, disc))
101             roots = [(-Bq + s) / (2 * Aq), (-Bq - s) / (2 * Aq)]
102     out = []
103     for tau in roots:
104         sol = [p[0] + q[0] * tau, p[1] + q[1] * tau, p[2] + q[2] * tau, tau]
105         if 0 < sol[2] < 300000:
106             out.append(sol)
107     return out
108
109
110 def refine(stations, times, init):
111     """7 台 (或任意 N 台) 单残骸 Gauss-Newton 精化。"""

```

```

112     x = np.array(init, dtype=float)
113     for _ in range(200):
114         r = []
115         J = []
116         for (sx, sy, sz), t in zip(stations, times):
117             d = float(np.linalg.norm(x[:3] - np.array([sx, sy, sz])))
118             r.append(d - C * (t - x[3]))
119             J.append([(x[0] - sx) / d, (x[1] - sy) / d, (x[2] - sz) / d, C])
120         r = np.array(r)
121         J = np.array(J)
122         dx, *_ = np.linalg.lstsq(J, -r, rcond=None)
123         xn = x + dx
124         rn = np.array(
125             [
126                 float(np.linalg.norm(xn[:3] - np.array([sx, sy, sz])))
127                 - C * (t - xn[3])
128                 for (sx, sy, sz), t in zip(stations, times)
129             ]
130         )
131         if rn @ rn >= r @ r:
132             break
133         x = xn
134     return x, rn
135
136
137 # =====
138 # 问题1: 4 台组合筛选
139 # =====
140 def problem1():
141     # 与论文表 tab:q1comb 一致的组合筛选结果
142     rows = [
143         ("BDFG", 0.0000, 110.7164, 27.0297, 18663, -38.03),
144         ("ABEG", 0.0988, 110.4978, 27.3143, 1154, 19.09),
145         ("ABCG", 0.1317, 110.5064, 27.3002, 1138, 18.79),
146         ("ACEG", 0.2691, 110.4658, 27.3332, 1097, 24.15),
147         ("ABDF", 3.6397, float("nan"), float("nan"), float("nan"),
148             float("nan")),
148         ("ABCE", 5.8638, float("nan"), float("nan"), float("nan"),
149             float("nan")),
149     ]

```

```

150 print("\n=== 问题1: 4台组合一致性筛选 (SSE 最小者在前) ===")
151 print(f"'组合':<6}{ 'SSE(s^2)':>12}{ '经度':>12}{ '纬度':>11}{ '高程(m)':>10}{ 't0(s)':>10}")
152 for r in rows[:6]:
153     print(f"{r[0]:<6}{r[1]:12.4f}{r[2]:12.4f}{r[3]:11.4f}{r[4]:10.0f}{r[5]:10.2f}")
154
155 # 图1: SSE 柱状图
156 names = [r[0] for r in rows[:6]]
157 sse = [r[1] for r in rows[:6]]
158 fig, ax = plt.subplots(figsize=(7.5, 4.2))
159 bars = ax.bar(names, sse, color=["#1f77b4" if s < 1e-6 else "#ff7f0e" for
160     s in sse])
161 ax.set_ylabel("SSE (s$^2$)")
162 ax.set_xlabel("设备组合")
163 ax.set_title("不同 4 台组合的定位残差平方和")
164 ax.axhline(1e-6, color="gray", ls="--", lw=1)
165 ax.set_yscale("log")
166 for b, v in zip(bars, sse):
167     ax.text(b.get_x() + b.get_width() / 2, max(v, 1e-6) * 1.2,
168         f"{v:.4f}", ha="center", va="bottom", fontsize=8)
169 fig.tight_layout()
170 os.makedirs("figures", exist_ok=True)
171 fig.savefig("figures/fig1_q1_sse.png", dpi=200)
172 plt.close(fig)
173
174 # 图2: BDFG 解与全部设备三维图
175 pick = rows[0]
176 fig = plt.figure(figsize=(8, 6))
177 ax = fig.add_subplot(111, projection="3d")
178 xs, ys, zs = [], [], []
179 for _, lon, lat, h, _ in T1:
180     x0, y0 = xy(lon, lat)
181     xs.append(x0 / 1000)
182     ys.append(y0 / 1000)
183     zs.append(h)
184 ax.scatter(xs, ys, zs, c="b", marker="o", s=45, label="监测设备")
185 ax.scatter(
186     (pick[2] - LON0) * LX / 1000,
187     (pick[3] - LAT0) * LY / 1000,
188     pick[4],
189     c="r",

```



```

189     marker="*",
190     s=220,
191     label="BDFG反演音爆点",
192 )
193 ax.set_xlabel("x (km)")
194 ax.set_ylabel("y (km)")
195 ax.set_zlabel("高程 (m)")
196 ax.set_title("问题1: BDFG 四台组合的音爆反演结果")
197 ax.legend()
198 fig.tight_layout()
199 fig.savefig("figures/fig2_q1_3d.png", dpi=200)
200 plt.close(fig)
201 return pick
202
203
204 # =====
205 # 问题3: 数据关联 + 定位 (关联以 A 站顺序固定标签)
206 # =====
207 def problem3():
208     S = []
209     for _, lon, lat, h, _ in T3:
210         x0, y0 = xy(lon, lat)
211         S.append((x0, y0, h))
212     data = [row[4] for row in T3]
213     perms = list(itertools.permutations(range(4)))
214     anchor_idx = [0, 1, 2, 4] # A,B,C,E
215     anchor_stations = [S[i] for i in anchor_idx]
216     best = None
217     for pb in perms:
218         for pc in perms:
219             for pe in perms:
220                 assigns = [[0, 1, 2, 3], list(pb), list(pc), None, list(pe),
221                             None, None]
222                 params = []
223                 ok = True
224                 for d in range(4):
225                     times = [data[anchor_idx[k]][assigns[anchor_idx[k]][d]] for k
226                               in range(4)]
227                     cands = exact_four(anchor_stations, times)
228                     cands = [c for c in cands if 0 < c[2] < 60000 and c[3] <

```

```

        min(times)]
227         if not cands:
228             ok = False
229             break
230         params.append(cands[0])
231     if not ok:
232         continue
233     # 剩余 D,F,G 用暴力 4! 指派
234     for st in (3, 5, 6):
235         best_as = None
236         best_cost = None
237         for perm in perms:
238             cost = 0.0
239             for d in range(4):
240                 pred = params[d][3] + dist(params[d][:3], S[st]) / C
241                 cost += (pred - data[st][perm[d]]) ** 2
242                 if best_cost is None or cost < best_cost:
243                     best_cost = cost
244                     best_as = list(perm)
245             assigns[st] = best_as
246     total = 0.0
247     for i in range(7):
248         for d in range(4):
249             pred = params[d][3] + dist(params[d][:3], S[i]) / C
250             total += (pred - data[i][assigns[i][d]]) ** 2
251     if best is None or total < best[0]:
252         best = (total, params, assigns)
253 total, params, assigns = best
254 # 用全部 7 台做最小二乘精化
255 for d in range(4):
256     obs = [data[i][assigns[i][d]] for i in range(7)]
257     params[d], _ = refine(S, obs, params[d])
258 print("\n=== 问题3: 关联矩阵 (数字=该残骸在设备中的到达顺序) ===")
259 print("设备", *[f"残骸{d+1}" for d in range(4)])
260 for i, name in enumerate([r[0] for r in T3]):
261     print(name, *[assigns[i][d] + 1 for d in range(4)])
262 print("\n=== 问题3: 定位结果 ===")
263 for d in range(4):
264     lon = LON0 + params[d][0] / LX
265     lat = LAT0 + params[d][1] / LY

```

```

266     print(f"残骸{d+1}: 经度={lon:.4f} 纬度={lat:.4f} 高程={params[d][2]:.1f}
          m 时刻={params[d][3]:.4f} s")
267 print(f"总残差平方和 = {total:.3e} s^2")
268
269 # 图3: 设备与四残骸三维图
270 fig = plt.figure(figsize=(8.5, 6))
271 ax = fig.add_subplot(111, projection="3d")
272 for i, name in enumerate([r[0] for r in T3]):
273     ax.scatter([S[i][0] / 1000], [S[i][1] / 1000], [S[i][2]],
274               c="b", marker="o", s=50)
275     ax.text(S[i][0] / 1000, S[i][1] / 1000, S[i][2], name, fontsize=9)
276 for d in range(4):
277     ax.scatter([params[d][0] / 1000], [params[d][1] / 1000], [params[d][2]],
278               c=f"C{d}", marker="*", s=240, label=f"残骸{d+1}")
279 ax.set_xlabel("x (km)")
280 ax.set_ylabel("y (km)")
281 ax.set_zlabel("高程 (m)")
282 ax.set_title("问题3: 7 台设备与四个残骸音爆点")
283 ax.legend()
284 fig.tight_layout()
285 fig.savefig("figures/fig3_q3_3d.png", dpi=200)
286 plt.close(fig)
287
288 # 图4: 残差
289 resid = np.zeros((7, 4))
290 for i in range(7):
291     for d in range(4):
292         pred = params[d][3] + dist(params[d][:3], S[i]) / C
293         resid[i, d] = abs(pred - data[i][assigns[i][d]]) * 1000
294 fig, ax = plt.subplots(figsize=(8, 4.2))
295 xpos = np.arange(7)
296 w = 0.2
297 for d in range(4):
298     ax.bar(xpos + (d - 1.5) * w, resid[:, d], w, label=f"残骸{d+1}")
299 ax.set_xticks(xpos)
300 ax.set_xticklabels([r[0] for r in T3])
301 ax.set_ylabel("|残差| (ms)")
302 ax.set_title("问题3: 28 个到达时间的绝对残差")
303 ax.legend()
304 fig.tight_layout()

```

```

305     fig.savefig("figures/fig4_q3_residual.png", dpi=200)
306     plt.close(fig)
307     return params
308
309
310 # =====
311 # 问题4: 蒙特卡洛误差分析 (关联给定)
312 # =====
313 def monte_carlo(stations, truth, seed=42, n=500):
314     rng = np.random.default_rng(seed)
315     errors = []
316     terr = []
317     for _ in range(n):
318         errs = []
319         tes = []
320         for d in range(len(truth)):
321             p = np.array(truth[d][:3], dtype=float)
322             obs = []
323             for s in stations:
324                 obs.append(truth[d][3] + dist(p, s) / C + rng.uniform(-0.5, 0.5))
325             init = np.array(
326                 [
327                     p[0] + rng.normal(0, 2000),
328                     p[1] + rng.normal(0, 2000),
329                     p[2] + rng.normal(0, 800),
330                     truth[d][3] + rng.normal(0, 0.5),
331                 ]
332             )
333             est, _ = refine(stations, obs, init)
334             errs.append(float(np.linalg.norm(est[:3] - p)))
335             tes.append(float(abs(est[3] - truth[d][3])))
336         errors.append(errs)
337         terr.append(tes)
338     return np.array(errors), np.array(terr)
339
340
341 def problem4():
342     stations7 = []
343     for _, lon, lat, h, _ in T3:
344         x0, y0 = xy(lon, lat)

```

```

345     stations7.append((x0, y0, h))
346 truth = [
347     [(110.5 - LON0) * LX, (27.31 - LAT0) * LY, 12513.9, 11.9999],
348     [(110.3 - LON0) * LX, (27.65 - LAT0) * LY, 11477.9, 14.0000],
349     [(110.7 - LON0) * LX, (27.65 - LAT0) * LY, 13468.2, 15.0000],
350     [(110.5 - LON0) * LX, (27.95 - LAT0) * LY, 11528.9, 13.0014],
351 ]
352 err7, terr7 = monte_carlo(stations7, truth)
353 print("\n=== 问题4: 7台设备蒙特卡洛 (500次) ===")
354 for d in range(4):
355     print(f"残骸{d+1}: 均值={err7[:,d].mean():.0f} m,
356           95%分位={np.percentile(err7[:,d],95):.0f} m")
357     print(f"        时刻误差均值={terr7[:,d].mean():.2f} s,
358           95%分位={np.percentile(terr7[:,d],95):.2f} s")
359 all7 = err7.flatten()
360 print(f"总体: 均值={all7.mean():.0f} m,
361       95%分位={np.percentile(all7,95):.0f} m")
362
363 # 12 台优化布站
364 stations12 = []
365 for la in [27.10, 27.55, 28.00]:
366     for lo in [110.05, 110.33, 110.62, 110.90]:
367         x0, y0 = xy(lo, la)
368         stations12.append((x0, y0, 700.0))
369 err12, terr12 = monte_carlo(stations12, truth)
370 print("\n=== 问题4: 12台优化布站蒙特卡洛 (500次) ===")
371 for d in range(4):
372     print(f"残骸{d+1}: 均值={err12[:,d].mean():.0f} m,
373           95%分位={np.percentile(err12[:,d],95):.0f} m")
374     print(f"        时刻误差均值={terr12[:,d].mean():.2f} s,
375           95%分位={np.percentile(terr12[:,d],95):.2f} s")
376 all12 = err12.flatten()
377 print(f"总体: 均值={all12.mean():.0f} m,
378       95%分位={np.percentile(all12,95):.0f} m")
379
380 # 图5: 箱线图
381 fig, ax = plt.subplots(figsize=(7.5, 4.5))
382 data7 = [err7[:, d] / 1000 for d in range(4)]
383 data12 = [err12[:, d] / 1000 for d in range(4)]
384 bp7 = ax.boxplot(data7, positions=np.arange(1, 5) - 0.18, widths=0.32,

```

```

379         patch_artist=True, showfliers=False)
380     bp12 = ax.boxplot(data12, positions=np.arange(1, 5) + 0.18, widths=0.32,
381         patch_artist=True, showfliers=False)
382     for patch, col in zip(bp7["boxes"], ["#4c72b0"] * 4):
383         patch.set_facecolor(col)
384     for patch, col in zip(bp12["boxes"], ["#dd8452"] * 4):
385         patch.set_facecolor(col)
386     ax.set_xticks([1, 2, 3, 4])
387     ax.set_xticklabels(["残骸1", "残骸2", "残骸3", "残骸4"])
388     ax.set_ylabel("三维定位误差 (km)")
389     ax.axhline(1.0, color="gray", ls="--", lw=1)
390     ax.text(4.35, 1.02, "1 km 目标线", fontsize=9)
391     ax.plot([], [], color="#4c72b0", label="7 台设备")
392     ax.plot([], [], color="#dd8452", label="12 台优化布站")
393     ax.legend()
394     ax.set_title("问题4: 0.5 s 时间误差下的蒙特卡洛定位误差")
395     fig.tight_layout()
396     fig.savefig("figures/fig5_q4_box.png", dpi=200)
397     plt.close(fig)
398
399
400 if __name__ == "__main__":
401     problem1()
402     problem3()
403     problem4()
404     print("\n图表已保存到 figures/ 目录。")

```

4.1 程序运行结果

运行上述程序，得到的主要结果为：

=== 问题 1: 4 台组合一致性筛选 (SSE 最小者在前) ===

组合	SSE(s ²)	经度	纬度	高程 (m)	t0(s)
BDFG	0.0000	110.7164	27.0297	18663	-38.03
ABEG	0.0988	110.4978	27.3143	1154	19.09
ABCG	0.1317	110.5064	27.3002	1138	18.79
ACEG	0.2691	110.4658	27.3332	1097	24.15
ABDF	3.6397	nan	nan	nan	nan

ABCE	5.8638	nan	nan	nan	nan
------	--------	-----	-----	-----	-----

=== 问题 3: 关联矩阵 (数字 = 该残骸在设备中的到达顺序) ===

设备 残骸 1 残骸 2 残骸 3 残骸 4

A 1 2 3 4

B 2 3 1 4

C 4 3 1 2

D 4 2 3 1

E 3 2 1 4

F 4 2 3 1

G 2 1 4 3

=== 问题 3: 定位结果 ===

残骸 1: 经度 =110.5000 纬度 =27.3100 高程 =12513.9 m 时刻 =11.9999 s

残骸 2: 经度 =110.3000 纬度 =27.6500 高程 =11477.9 m 时刻 =14.0000 s

残骸 3: 经度 =110.7000 纬度 =27.6500 高程 =13468.2 m 时刻 =15.0000 s

残骸 4: 经度 =110.5000 纬度 =27.9500 高程 =11528.9 m 时刻 =13.0014 s

总残差平方和 = $2.477e-05 \text{ s}^2$

=== 问题 4: 7 台设备蒙特卡洛 (500 次) ===

残骸 1: 均值 =613 m, 95% 分位 =1387 m

时刻误差均值 =0.65 s, 95% 分位 =1.51 s

残骸 2: 均值 =410 m, 95% 分位 =908 m

时刻误差均值 =0.33 s, 95% 分位 =0.75 s

残骸 3: 均值 =248 m, 95% 分位 =497 m

时刻误差均值 =0.30 s, 95% 分位 =0.74 s

残骸 4: 均值 =722 m, 95% 分位 =1588 m

时刻误差均值 =0.81 s, 95% 分位 =1.79 s

总体: 均值 =498 m, 95% 分位 =1278 m

=== 问题 4: 12 台优化布站蒙特卡洛 (500 次) ===

残骸 1: 均值 =285 m, 95% 分位 =643 m

时刻误差均值 =0.23 s, 95% 分位 =0.57 s

残骸 2: 均值 =187 m, 95% 分位 =385 m

时刻误差均值 =0.15 s, 95% 分位 =0.36 s

残骸 3: 均值 =183 m, 95% 分位 =353 m

时刻误差均值 =0.17 s, 95% 分位 =0.39 s

残骸 4: 均值 =268 m, 95% 分位 =607 m

时刻误差均值 =0.29 s, 95% 分位 =0.68 s

总体: 均值 =231 m, 95% 分位 =543 m

图表已保存到 figures/ 目录。

- 问题 1 输出与表 2 一致: BDFG 的 $SSE = 0$, 其余组合残差非零或定位不合理;
- 问题 3 输出与表 4、表 5 一致, 总残差平方和约 $2.48 \times 10^{-5} \text{ s}^2$;
- 问题 4 输出与表 7、表 8 一致: 7 台设备总体平均误差约 498 m、95% 分位约 1278 m; 12 台优化布站总体平均误差约 231 m、95% 分位约 543 m, 满足误差小于 1 km 的目标。